Phantom Packets — Technical Write-Up

NETWORK FORENSICS & PACKET CAPTURE ANALYSIS

Challenge: Phantom Packets	**Target IP:** 172.28.0.10	**Category:** Network Forensics / Crypto
Difficulty: Hard	**Protocol:** DNS / TLS / Git	**Status:** Solved (2/2 Flags)

EXECUTIVE SUMMARY

An analysis of the `phantom_packets.pcapng` capture file revealed a 4-stage incident targeting the internal host `172.28.0.10`. The attacker exfiltrated an RSA private key using DNS tunneling, leveraged non-PFS static RSA cipher suites to decrypt captured HTTPS traffic, extracted an internal Git repository (`nexline-deploy.git`), and retrieved sensitive credentials from active configuration files and unlinked Git history objects.

DETAILED INVESTIGATION LIFECYCLE

Stage 1: DNS Tunneling & Private Key Exfiltration

Initial inspection showed suspicious, high-frequency DNS queries targeting `status.update-svc-cdn.com`. The subdomains consisted of sequential 4-digit index prefixes followed by hex-encoded strings.

```
0001.308202473082012f...status.update-svc-cdn.com
0002.020100300d06092a...status.update-svc-cdn.com
```

Reassembling the hex payload sequentially yields an ASCII **RSA Private Key** (PEM format) belonging to the server's TLS certificate.

Stage 2: Static RSA TLS Decryption

With the extracted private key, encrypted HTTPS traffic directed to `172.28.0.10:443` was analyzed. Because the server negotiated static RSA cipher suites without Perfect Forward Secrecy (PFS), importing the private key into Wireshark allowed full plaintext decryption of the TLS session streams.

Stage 3: Git Repository Extraction

Inside the decrypted HTTP stream, a `git clone` transaction for repository `nexline-deploy.git` was identified. Extracting the raw packfile payload and running `git unpack-objects` successfully reconstructed the working tree and commit database locally.

Stage 4: Flag & Credential Recovery

Forensic inspection of the reconstructed repository yielded two flag segments:

- **Part 1 (Active Branch):** Inspected `deploy/staging-vars.conf` containing the active staging token.
- **Part 2 (Dangling Git History):** Querying unlinked commit objects via `git fsck --lost-found` revealed a deleted file, `config/credentials.bak`, containing the API secret.

FLAG RECOVERY SUMMARY

Part 1 (Staging Vars): `TCTT2026_pt1{5b9437e03df7b252}`

Part 2 (Git History): `TCTT2026_pt2{9562ad4705a54963}`

Combined Flag: `TCTT2026_pt1{5b9437e03df7b252}TCTT2026_pt2{9562ad4705a54963}`

REMEDIATION & RISK MITIGATION

Vulnerability	Impact	Recommended Remediation
DNS Tunneling Exfiltration	Data leakage via outbound DNS queries	Implement DNS filtering, inspect high-entropy subdomain requests, and enforce strict egress firewall policies.
Static RSA Cipher Suites	Retroactive TLS traffic decryption	Disable non-PFS key exchanges (RSA); enforce Ephemeral Diffie-Hellman (ECDHE) cipher suites.
Secrets in Git History	Credential exposure in deleted commits	Use tools like `git-filter-repo` or BFG Repo-Cleaner to purge historical secrets; rotate compromised keys immediately.